كيف تشغّل الأدوات كمبتدئ

إطار Cerberus · دليل تعليمي محلي · 2 نوفمبر 2026 (توقيت القاهرة)

> **ابدأ من هنا:** هذه الأوامر تشغّل معمل تجريبي مؤقتاً على جهازك. لا تزور موقعاً أو جهازاً تحدده أنت. المعمل عمداً فيه ثغرات، ويستمع فقط على `127.0.0.1` حتى ينتهي الأمر.

ما المتاح؟

الأداة	في هذا الدليل	ما الذي يثبته التشغيل الناجح
Nmap	حقيقي مثبت على جهازك Nmap	يجد المنفذ TCP المفتوح المؤقت في المعمل.
sqlmap	حقيقي مثبت على جهازك sqlmap	يكتشف حقن SQL المقصود في المعمل.
Metasploit	**محاكاة** وليس البرنامج الحقيقي	المحوّل يرسل ويقرأ استجابة MessagePack مزيفة. المحاكي لا يفحص شيئاً.

في 2 نوفمبر 2026، شغّلت اختبار Linux منفصل **الأدوات الحقيقية الثلاث في نفس الوقت** في محركات منفصلة: Nmap وsqlmap على مواقع عامة تجريبية مسموح بها، وخادم Metasploit الحقيقي على مستمع مؤقت على جهازك. ذلك الاختبار استخدم `msfrpcd` حقيقياً. أوامر المبتدئ في هذا الدليل ما زالت تستخدم المعمل المحلي، وMetasploit فيه محاكى. نتائج Nmap لا تنتقل تلقائياً إلى Metasploit.

واجهة التطبيق، وواجهة API، ووكيل الذكاء الاصطناعي **لا** تشغّل هذه الوحدات حتى الآن؛ ربطها مرحلة لاحقة.

اختر ما تحتاجه

- لـ **وحده Nmap**: ثبّت متطلبات Python للخلفية وNmap.
- لـ **وحده sqlmap**: ثبّت متطلبات Python للخلفية وsqlmap.
- لـ **محاكاة Metasploit وحدها**: تحتاج متطلبات Python للخلفية فقط. لا يلزم تحميل Metasploit بحجم 1 جيغابايت.
- لـ **all** أو **parallel**: تحتاج Nmap حقيقياً وsqlmap حقيقياً؛ وMetasploit ما زال محاكى.
- لتشغيل **عرض واحد** يجرّب كل الخطوات: انظر القسم 4C. التشغيل على المواقع العامة اختياري وموضح هناك.

لا تحتاج تشغيل تطبيق الويب أو PostgreSQL أو Redis أو خادم Metasploit حقيقي لهذا المعمل. المعمل يعرض النتائج ولا يحفظها في قاعدة بيانات التطبيق.

1. الإعداد لمرة واحدة: MacBook / zsh

أ. افتح المجلد الصحيح

في VS Code اختر **File → Open Folder**، ثم اختر مجلد المشروع الذي يحتوي على `backend` و`docs` و`scripts` ، ثم اختر **Terminal → New Terminal**. كل الأوامر في هذا الدليل تبدأ من جذر المشروع. لا تكتب علامة `$` نفسها.

ب. جهّز بيئة Python الخاصة بالمشروع

المتطلبات مثبتة للإصدارين **Python 3.11 أو 3.12**، والأفضل 3.12. إذا احتجت، ثبّته بـ Homebrew: `brew install python@3.12`. ثبّت Homebrew أولاً من brew.sh إذا لم يكن الأمر `brew` موجوداً.

إذا كان المجلد `backend/.venv` موجوداً فعلاً، **لا تحذفه ولا تعد إنشاءه**؛ تخطَّ السطر الأول. ثبّت مكتبات المشروع فيه:

```
python3.12 -m venv backend/.venv
./backend/.venv/bin/python -m pip install -r backend/requirements.txt
```

ج. ثبّت الأدوات الحقيقية وتحقق منها

```
brew install nmap sqlmap
nmap --version
sqlmap --version
```

ثبّت الأداة التي تريدها فقط إذا لم تكن تشغّل الاثنتين. إذا ظهر أن الأمر غير موجود بعد التثبيت، أعد فتح طرفية VS Code واتبع قسم استكشاف الأخطاء الخاص بالمسار PATH.

د. الإعدادات والمفتاح السري

المعمل يفعّل التكاملات المختارة **في الذاكرة لذلك التشغيل فقط**؛ لا يعدّل أي ملف `.env`. إذا لم يكن `SECRET_KEY` مضبوطاً، يولّد المعمل مفتاحاً عشوائياً مؤقتاً خاصاً بالعملية. إذا كان المفتاح مضبوطاً لكنه قصير، فأصلح الإعداد بدلاً من ذلك. للتطبيق الكامل، اضبط مفتاحاً عشوائياً خاصاً ودائماً بطول 32 حرفاً على الأقل في `backend/.env`؛ لا تشاركه ولا تستخدم المثال الموجود في الدليل.

2. الإعداد لمرة واحدة: Windows PowerShell

افتح مجلد المشروع في VS Code، ثم افتح طرفية **PowerShell** في جذر المشروع. استخدم Python 3.11 أو 3.12؛ المثال يستخدم 3.12. إذا كان `backend\.venv` موجوداً، تخطَّ السطر الأول ولا تحذفه.

```
py -3.12 -m venv backend\.venv
.\backend\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

تثبيت Nmap

حمّل المثبّت الرسمي لويندوز من nmap.org/download وشغّله. أعد فتح VS Code أو الطرفية حتى يتحدّث المسار PATH، ثم تحقق:

```
nmap --version
```

إذا لم يُعثر عليه، ابحث عن الملف `nmap.exe` المثبّت، واضبط `NMAP_PATH` على مساره **الفعلي** الكامل لهذه الطرفية؛ انظر قسم استكشاف الأخطاء.

تثبيت sqlmap

ثبّت Git for Windows إذا فشل الأمر `git --version`. الأوامر التالية تنزّل مستودع sqlmap الرسمي في مجلد جديد داخل مجلدك الشخصي. إذا كان المجلد موجوداً ويحتوي `sqlmap.py`، تخطَّ الاستنساخ؛ وإلا اختر مساراً آخر فارغاً.

```
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git "$HOME\cerberus-sqlmap"
$env:SQLMAP_PATH = "$HOME\cerberus-sqlmap\sqlmap.py"
.\backend\.venv\Scripts\python.exe "$env:SQLMAP_PATH" --version
```

`SQLMAP_PATH` يُضبط فقط للجلسة الحالية في PowerShell؛ كرر السطر الثاني في طرفية جديدة. لمحاكاة Metasploit وحدها لا تحتاج تثبيت Nmap ولا sqlmap.

3. شغّل أداة واحدة في كل مرة

أكمل الإعداد أولاً لنظامك. شغّل كل أمر من جذر المشروع، وانسخ السطر الخاص بنظام تشغيلك. كل تشغيل يبدأ معمله المحلي ويوقفه بنفسه.

ملف تنفيذي حقيقي: Nmap

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py nmap
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py nmap
```

المتوقع: السطر `PASS: real Nmap found the local lab's open port` متبوعاً برقم منفذ يتغير في كل مرة.

ملف تنفيذي حقيقي: sqlmap

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py sqlmap
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py sqlmap
```

المتوقع: السطر `PASS: real sqlmap detected` مع معامل `id` المقصود أنه ضعيف. هذه نتيجة إيجابية **للمعمل التجريبي فقط**، وليست دليلاً عن تطبيقك أنت.

استجابة محاكاة فقط: Metasploit

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py metasploit
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py metasploit
```

المتوقع: كلمة `SIMULATED` و`PASS` لقراءة الاستجابة المزيفة. لم يتم اختبار `msfconsole` أو `msfrpcd` أو أي فحص Metasploit حقيقي.

4. شغّلها معاً

الخيار A: خطة واحدة متسلسلة `all`

يشغّل **المحاكى Metasploit ← sqlmap ← Nmap**، واحدة بعد الأخرى، في محرك واحد. لا تعمل في اللحظة نفسها. المنافذ التي يكتشفها Nmap لا تُمرَّر تلقائياً إلى وحدة Metasploit.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py all
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py all
```

الخيار B: متوازٍ ومنفصل `parallel`

هذه التشغيلات الثلاثة المستقلة تتداخل في الوقت على **نفس الخادم التجريبي المحلي**. لكل واحدة سياق منفصل؛ ولا تُشارَك النتائج. هذا عرض للتشغيل المتزامن، وليس سلوك المجدول العادي.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py parallel
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py parallel
```

الخيار C: عرض واحد يشغّل كل الخطوات

يشغّل المعمل المحلي (Nmap وsqlmap وMetasploit المحاكى معاً)، ثم الاختبارات الآلية التي لا تحتاج قاعدة بيانات، ويطبع ملخصاً. كل خطوة لها تسمية.

Mac / zsh

```
export SQLMAP_PATH="$(command -v sqlmap)"
./backend/.venv/bin/python scripts/demo/run_everything.py
```

Windows / PowerShell

```
$env:SQLMAP_PATH = "$HOME\cerberus-sqlmap\sqlmap.py"
.\backend\.venv\Scripts\python.exe scripts\demo\run_everything.py
```

معنى التسميات: **LOCAL LAB** هدف مؤقت على جهازك؛ **SIMULATED** استجابة Metasploit المزيفة؛ **TESTS** تشغيل الاختبارات الآلية. يطبع الملخص PASS أو FAIL أو SKIP لكل خطوة، وأي خطوة متخطاة تذكر سببها دائماً. قائمة الأوامر الكاملة موجودة في ملف "VS Code tools test".

اختياري: التشغيل على مواقع عامة

هذه التشغيلات تزور مواقع عامة مسموحاً بها، لذلك استخدمها فقط عندما تحتاج نتيجة جديدة. كل واحدة تحتاج `--confirm-public-tests`، وكل واحدة تُحسب ضمن حد يومي (Nmap حوالي 10 مرات في 24 ساعة، وsqlmap 6 مرات في 24 ساعة). وهي تحفظ النتائج في `docs/real_results.json`. ولا تقبل هدفاً منك.

- **Nmap على `scanme.nmap.org` فقط:** `public_tool_results.py nmap --confirm-public-tests`
- **sqlmap على معامل `id` في SekuMart فقط:** `public_tool_results.py sqlmap --confirm-public-tests`

- **الثلاثة معاً، مع Metasploit الحقيقي على الجهاز:** اضبط `MSF_RPC_PASSWORD`، وتأكد أن `msfrpcd` يعمل، ثم شغّل `scripts/live_tests/run_all_tools.py --confirm-public-tests`.

استخدم الشكل `./backend/.venv/bin/python` على Mac، والشكل `.\backend\.venv\Scripts\python.exe` على Windows، كما في الأوامر السابقة.

المخرجات المتوقعة

التشغيل الناجح للأمر `all` أو `parallel` يعرض عنواناً محلياً، وثلاثة أسطر `PASS`، وتسمية `SIMULATED` لـ Metasploit، ثم `DONE`. المنفذ يتغير في كل مرة. رسالة `FAILED` أو تبعية مفقودة أو نتيجة متخطاة أو نتيجة `ERROR` **ليست** فحصاً ناجحاً. انتظر حتى ينتهي الأمر؛ sqlmap الحقيقي قد يستغرق وقتاً.

عند الانتهاء الطبيعي، تتوقف الخوادم المحلية ويُنظَّف ناتج sqlmap المؤقت. التقرير لا يُحفظ في PostgreSQL ولا يظهر في التطبيق. الضغط على Ctrl+C يطلب التنظيف؛ أما الإيقاف القسري فقد يترك ملفات مؤقتة.

5. استكشاف الأخطاء

إذا رأيت...	جرّب هذا
`No such file` لمسار Python	افتح جذر المشروع في VS Code، وتأكد أن `backend/.venv` موجود، واستخدم الأمر الخاص بنظامك.
`Missing Python package`	شغّل تثبيت pip باستخدام Python الخاص بـ `.venv` للمشروع، وليس Python آخر.
`Missing required tool(s)`	ثبّت الأداة الحقيقية المذكورة. `all` و`parallel` يحتاجان الاثنتين؛ ومحاكاة Metasploit لا تحتاج أياً منهما.
`nmap` أو `sqlmap` غير موجود	أعد فتح الطرفية، وتحقق من الملف والمسار PATH، أو اضبط المتغيرات أدناه. متغير البيئة الموروث يتجاوز `backend/.env`.
`SECRET_KEY` قصير جداً	أصلح القيمة الموجودة أو ملف `backend/.env` الخاص؛ استخدم 32 حرفاً عشوائياً على الأقل. لا تشاركه.
فشل sqlmap أو انتهت مهلته	تحقق من إصداره ومساره، وتحقق من برامج الحماية على جهازك، ثم أعد المحاولة. الفحص الفاشل أو غير الحاسم ليس دليلاً على أن الجهاز سليم.
`msfconsole: command not found`	**لا يلزم** لهذا المعمل. المحاكاة لا تتحقق من تثبيت Metasploit حقيقي.
تشغيل Metasploit حقيقي منفصل تنتهي مهلته عند إنشاء أول console	أول console يكون عادة سريعاً (0.48 ثانية على جهاز الاختبار). القيمة الافتراضية `MSF_RPC_TIMEOUT=60`؛ وإذا كانت قيمة قديمة في `.env` خاص تتجاوزها. هذا الإعداد لا يلزم لمحاكاة المبتدئ.
`--target` مرفوض	هذا المشغّل لا يقبل هدفاً مخصصاً عمداً؛ يستخدم معمله المحلي فقط.

إذا لم تكن الأدوات في PATH

Mac / zsh: بعد التثبيت بـ Homebrew، تضبط هذه الأوامر المسارات لهذه الطرفية:

```
export NMAP_PATH="$(command -v nmap)"
export SQLMAP_PATH="$(command -v sqlmap)"
```

Windows / PowerShell: استبدل المثال الأول بالمسار الفعلي لملف `nmap.exe`؛ والمثال الثاني يستخدم نسخة sqlmap من خطوة الإعداد:

```
$env:NMAP_PATH = 'C:\Program Files (x86)\Nmap\nmap.exe'
$env:SQLMAP_PATH = "$HOME\cerberus-sqlmap\sqlmap.py"
```

تحقق بـ `& $env:NMAP_PATH --version` و`Test-Path $env:SQLMAP_PATH`. هذه المتغيرات تنطبق على الطرفية الحالية فقط. لا تفترض أن مسار Nmap في المثال يطابق جهازك.

6. الملفات والمجلدات المهمة

كل المسارات نسبية إلى جذر المشروع. ملف `.env` خاص؛ و`.env.example` قالب وليس مخزناً للأسرار.

المسار	لماذا يهم
`scripts/live_tests/beginner_tools_lab.py`	أوضاع المبتدئ: `nmap` و`sqlmap` و`metasploit` المحاكى و`all` و`parallel`. بدون هدف مخصص.
`scripts/live_tests/sqlmap_framework_smoke.py`	معمل محلي أوسع وخادم RPC وهمي يعيد المشغّل نفسه استخدامه.
`scripts/demo/run_everything.py`	عرض واحد يشغّل المعمل المحلي والاختبارات، واختيارياً التشغيلات العامة.
`scripts/demo/run-vscode-tools-.ps1` / `test.sh`	ملفات تشغيل مختصرة للعرض على Mac وWindows.
`scripts/live_tests/public_tool_results.py`	تشغيلات عامة اختيارية لـ Nmap أو sqlmap فقط. تحتاج `--confirm-public-tests`.
`scripts/live_tests/run_all_tools.py`	تشغيل متزامن اختياري للأدوات الثلاث، مع Metasploit الحقيقي على الجهاز. يحتاج `--confirm-public-tests`.
`backend/requirements.txt` و`backend/.venv/`	متطلبات الخلفية وبيئة Python المحلية. بيئة `.venv` غير محفوظة في Git.
`backend/.env.example` / `backend/.env`	الإعدادات الخاصة / مثال الإعدادات. المتغيرات المصدَّرة لها الأولوية. لا تحفظ الأسرار في Git.
`backend/app/ai_engine/pentest/tools/`	المحوّلات الخارجية: `nmap_adapter.py` و`sqlmap_adapter.py` و`metasploit_adapter.py`، وسجل الأدوات.
`backend/app/ai_engine/pentest/modules/tools/`	وحدات الإطار: `nmap_scanner.py` و`sqlmap_scanner.py` و`metasploit_scanner.py`.
`backend/app/ai_engine/pentest/core/`	السياق والنطاق وفحوص الأمان وعقود النتائج والأدلة.
`backend/app/ai_engine/pentest/engine/`	اكتشاف الوحدات، ومحرك التنفيذ، والمجدول **المتسلسل**.
`backend/app/ai_engine/pentest/integration/`	دوال مساعدة صريحة لتحويل المحركات وحفظ النتائج؛ لا تُستدعى تلقائياً من الواجهة.
`backend/tests/unit/`	اختبارات الإطار والأدوات، بما فيها `test_sqlmap_integration.py` و`test_beginner_tools_lab.py`.
`docs/SQLMAP_INTEGRATION.md`	تفاصيل أكثر عن محوّل sqlmap وقيود النطاق والتحقق.
`docs/HOW_TO_RUN_THE_TOOLS_AS_BEGINNER.md`	المصدر القابل للتعديل لهذا الدليل. ملف PDF المرفق هو النسخة القابلة للطباعة.
`docs/real_results.json`	النتائج المحفوظة، مصنّفة إلى عامة ومعمل محلي ومحاكاة. ملف PDF للنتائج الحقيقية مبني منها.
`docs/all_tools_result/run-all-tools.json`	ناتج تشغيل الأدوات الثلاث في 2 نوفمبر.
`scripts/render_beginner_tools_guide.py`	يعيد بناء ملف PDF للدليل من مصدر Markdown.

يحتوي الإطار أيضاً على وحدات مدمجة للويب والشبكة وفحص بيانات الاعتماد داخل `pentest/modules/`. هذه الوحدات **منفصلة عن التكاملات الخارجية الثلاث**، ولا تدخل في وضع `all` في هذا الدليل.

7. الحدود والأمان والصيانة

ما الذي لا يثبته المعمل

- واجهة التطبيق وواجهة API ووكيل الذكاء الاصطناعي غير مرتبطة باستدعاء هذا الإطار. تفعيل إعداد لا يضيف زراً ولا يبدأ فحصاً.
- خدمة Metasploit المزيفة تعيد بيانات اختبار؛ لا تشغّل وحدة Metasploit حقيقية ولا تتحقق منها. اختبار Linux منفصل على الـ loopback تحقق من تكامل MessagePack الحقيقي؛ وكان اختباراً محلياً وليس عاماً.
- فحوص الأمان تطابق النطاق بطريقة منظمة والمنافذ الافتراضية. Metasploit يسمح فقط بمُماسح المنافذ TCP المراجَع وبمنافذ محدودة؛ Nmap NSE يسمح فقط بـ `banner` و`ssl-cert`. الوحدات غير المراجعة واختيارات الفئات أو البطاقات تُرفض، حتى عند تأثير FULL.
- كل الوحدات ترفض `testing_window` مضبوطة صراحةً إلى أن يُنفَّذ معناها. فحوص أسماء المضيفين مع استثناءات IP/ العادي لأسماء المضيفين غير مثبَّت. فترة الاختبار القانونية لا تُحمَّل تلقائياً DNSمرفوضة، و CIDR**لا** تعامل الإطار على أنه جاهز للإنتاج.
- افحص الأنظمة التي لديك تفويض كتابي صريح بفحصها فقط، مع أهداف ومنافذ وحدود تأثير وتوقيت معتمدة. الأوامر هنا لا تقبل أبداً هدفاً حقيقياً.

إذا وصلت لاحقاً بـ Metasploit daemon حقيقي

هذا اختياري: معمل المبتدئ أعلاه يستخدم المحاكي دائماً. يجب تشغيل `msfrpcd` حقيقي بشكل منفصل، باسم مستخدم RPC هو `msf` وكلمة مرور خاصة قوية. اضبط `MSF_RPC_HOST` و`MSF_RPC_PORT` و`MSF_RPC_PASSWORD` المقابلة في بيئتك الخاصة، واضبط `MSF_ENABLED=true`. التكامل يتكلم MessagePack وليس XML-RPC. يدعم فقط الوحدة المراجَعة `auxiliary/scanner/portscan/tcp` والمنافذ المحدودة المعتمدة.

أبقِ الخادم على loopback عند استخدام `MSF_RPC_SSL=false`؛ الاتصالات البعيدة تحتاج TLS موثوقاً، وربما `MSF_RPC_CA_BUNDLE` لشهادة CA موثوقة. الخادم يمنح تحكماً قوياً في Metasploit: لا تعرضه للعامة ولا تشارك كلمة مروره. القيمة الافتراضية `MSF_RPC_TIMEOUT` هي 60 ثانية، لكن قيمة قديمة في `.env` خاص تتجاوزها. تثبيت Metasploit أو تفعيله لا يربط واجهة التطبيق أو API. Windows أو macOS وليس Linux أو وكيل الذكاء الاصطناعي. تشغيلات الخادم الحقيقي في هذه الإصدار تمت على

مهم: اختبارات الخلفية العامة قد تحذف الجداول

معمل المبتدئ **لا** يحتاج PostgreSQL ولا يشغّل مجموعة اختبارات الخلفية العامة. الملف `backend/tests/conftest.py` يُنشئ الجداول ويحذفها، ويحسب عنوان قاعدة الاختبار باستبدال نص داخل عنوان قاعدة بيانات التطبيق؛ وهذا قد يختار قاعدة خاطئة أو يغيّر بيانات الدخول. لا تشغّل المجموعة العامة على قاعدة بياناتك الحقيقية. أولاً أنشئ قاعدة بيانات منفصلة مؤقتة، وعزل عنوان قاعدة البيانات كاملاً وRedis، وتحقق من عنوان الاختبار الناتج. التحقق في هذا التقرير استخدم قاعدة مؤقتة وحذفها بعد ذلك.

الحفاظ على هذا الدليل محدثاً

عندما تُضاف أداة خارجية أو تُحذف أو يتغير سلوكها، يجب على المساهم تحديث جدول الأدوات والمتطلبات والأمثلة الفردية والمجمعة والقيود وقائمة الملفات والاختبارات؛ وإضافة مثال محلي آمن إذا أمكن. إذا لم يمكن التحقق، قل ذلك صراحة. أعد بناء ملف PDF وافحص كل صفحة في **التغيير نفسه**. هناك اختبار يتحقق من علامات الأدوات المسجلة ومن بصمة المصدر داخل ملف PDF، لكنه لا يستطيع إثبات أن التعليمات صحيحة.

المطور: أعد بناء ملف PDF القابل للطباعة

ثبّت متطلبات التطوير، وتبعيات WeasyPrint على نظام التشغيل إذا لزم. من جذر المشروع:

Mac / zsh

```
./backend/.venv/bin/python -m pip install -r backend/requirements-dev.txt
./backend/.venv/bin/python scripts/render_beginner_tools_guide.py
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe -m pip install -r backend\requirements-dev.txt
.\backend\.venv\Scripts\python.exe scripts\render_beginner_tools_guide.py
```

المخرج المعتمد هو `docs/how to run the tools as beginner.pdf`. افحص كل صفحة وتحقق من الأوامر على نظام التشغيل المعني قبل النشر.

8. التحقق والروابط المفيدة

ما الذي تم فحصه في هذه النسخة

في 2 نوفمبر 2026 (بيئة Linux) نجحت هذه الاختبارات: تشغيل Nmap منفرداً، ومحاكاة Metasploit منفردة، وتشغيلا `all` و`parallel` المحليان، والخطوات المحلية لعرض `run_everything`، و228 اختباراً آلياً لا تحتاج قاعدة بيانات. مجموعة الخلفية الكاملة نجحت باختبارات 385 على قاعدة بيانات مؤقتة. خطة المعمل المتسلسلة والمحركات المنفصلة المتزامنة تستخدم نقطة Metasploit **محاكاة**. وبشكل منفصل، أجرى اختبار Linux في 2 نوفمبر تشغيلاً حقيقياً لـ Nmap وsqlmap وMetasploit في الوقت نفسه (انظر ملف PDF لأداة الأدوات الثلاث وملف PDF للنتائج الحقيقية للمخرجات المؤرخة). نتائج Nmap لا تنتقل تلقائياً إلى Metasploit.

تعليمات Mac وWindows كُتبت لهذين النظامين لكنها **لم تُنفَّذ على macOS أو Windows** في هذه النسخة.

المعمل لا يحتاج التطبيق كاملاً. يتحقق من نتائج يمكن التعرف عليها، وليس من مجرد نجاح العملية.

فحص أعمق اختياري

اختبار sqlmap التجريبي الأوسع يغطي GET وPOST وإعادة التوجيه ومعالجة المعاملات والمحركات المتزامنة. يحتاج مفتاح سري مضبوطاً للخلفية وsqlmap حقيقياً؛ ويستخدم Nmap الحقيقي إذا كان متاحاً، وMetasploit **مزيفاً**. ولا يحفظ النتائج.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/sqlmap_framework_smoke.py
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\sqlmap_framework_smoke.py
```

يقرأ السكربت `backend/.env` أو متغيرات البيئة ويحتاج مفتاحاً سرياً موجوداً. إذا كان Nmap غير متوفر، يتخطاه؛ راجع المخرجات.

المراجع الرسمية

- تنزيلات Nmap ومثبّت Windows · Homebrew على Nmap
- مستودع sqlmap الرسمي · Homebrew على sqlmap
- وثائق Rapid7 لـ Metasploit RPC
- معطّل Homebrew على Metasploit cask؛ استخدم إرشادات تثبيت Rapid7 إذا احتجت daemon حقيقياً.